Design principles

CS2103T · Lecture 6

Demo course material, written for OpenPapr

Why design principles?

- Code is changed far more often than it is written from scratch
- Good design keeps each change small and local

Coupling

- Coupling is how much one component depends on others
- High coupling: a change in one class forces changes in others
- Aim for low coupling

Cohesion

- Cohesion is how strongly the parts of one component belong together
- High cohesion: a class does one job, and does it well
- Aim for high cohesion

SOLID at a glance

- S: Single Responsibility Principle
- O: Open-Closed Principle
- L: Liskov Substitution Principle
- I: Interface Segregation Principle
- D: Dependency Inversion Principle

Single Responsibility

- A class should have only one reason to change
- Example: keep Storage and Parser apart, not one class that does both

Open-Closed

- Open for extension, closed for modification
- Add new behaviour by adding code, not by editing code that works
- Usually done with polymorphism: interfaces and abstract classes

Liskov Substitution

- A subclass object must work wherever its superclass is expected
- A subclass must not demand more or promise less than its superclass
- Classic violation: Square extends Rectangle

Interface Segregation

- No client should be forced to depend on methods it does not use
- Prefer several small interfaces to one large one

Dependency Inversion

- High-level modules should not depend on low-level modules; both should depend on abstractions
- Example: Logic depends on a Storage interface, not on JsonStorage

Summary

- Low coupling, high cohesion
- SOLID: five principles for code that is easy to change
- Next: design patterns